

CRUD de productos - Angular

Objetivo

Desarrollar una aplicación en Angular 17 utilizando arquitectura modular, componentes y servicios para gestionar productos en memoria.

La aplicación deberá permitir:

- Crear productos.

- Visualizar productos.

- Eliminar productos.

- (Opcional) Actualizar productos.

El objetivo principal es practicar:

- Servicios.

- Inyección de dependencias.

- Comunicación entre componentes.

- Organización de proyectos Angular.

- Manejo de datos en memoria.

Restricciones importantes

✗ NO usar:

- Standalone Components

- Signals

- Observables

- HttpClient

- APIs

- Backend

LocalStorage

Librerías externas

Inteligencia artificial para generar el proyecto completo

✓ DEBE utilizarse:

Angular modular.

Componentes.

Servicios.

TypeScript.

Datos en memoria.

⚠ Cualquier implementación utilizando tecnologías o enfoques no vistos en clase tendrá penalización.

Descripción general de la aplicación

La aplicación tendrá una sola vista principal dividida en dos secciones:

Izquierda	Derecha
Lista de productos	Formulario

Diseño esperado

Sección izquierda — Lista de productos

Debe mostrar:

Un mensaje cuando no existan productos.

Una lista de tarjetas cuando existan productos.

Caso sin productos

Cuando no haya productos creados, debe mostrarse:

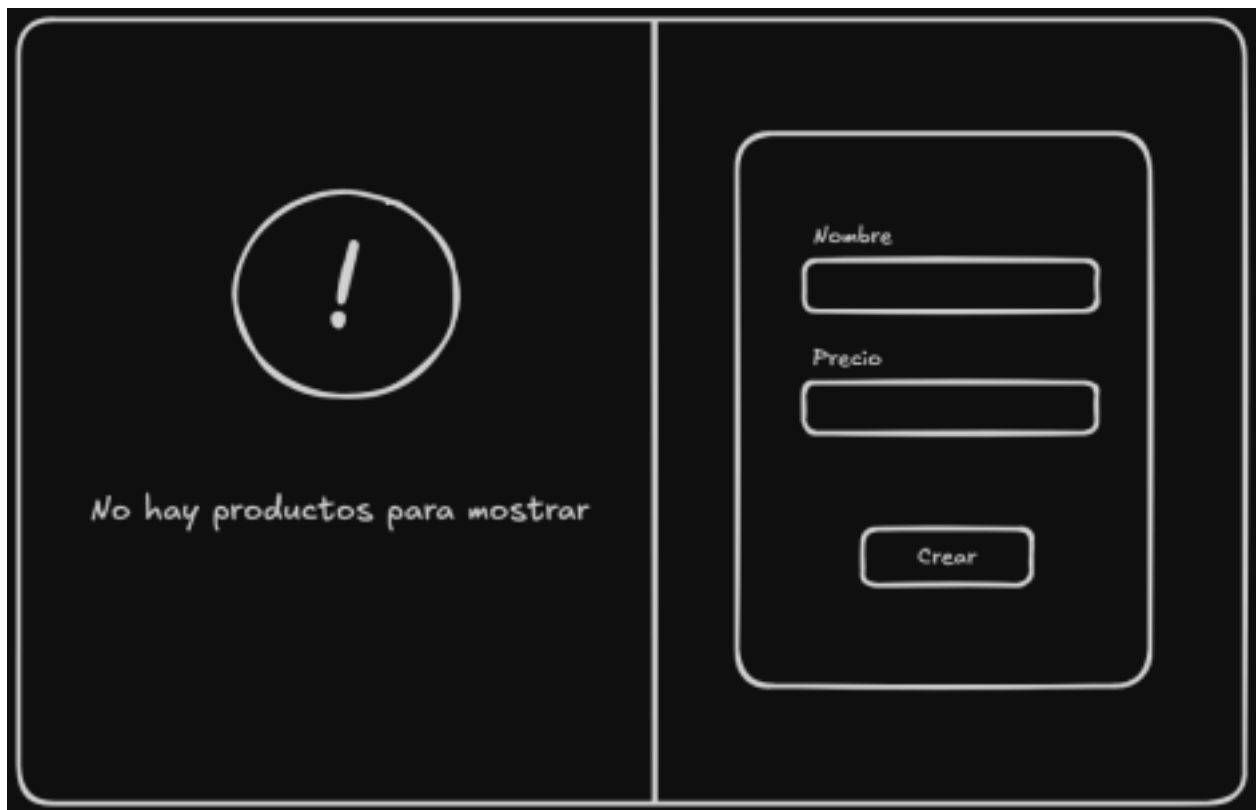
Un ícono de advertencia.

El texto:

No hay productos para mostrar

El mensaje debe estar centrado visualmente.

Ejemplo visual:



Caso con productos

Cuando existan productos:

Debe mostrarse una lista de tarjetas.

Cada tarjeta debe contener:

Nombre del producto.

Precio del producto.

Un elemento para eliminar:

botón,

ícono,

texto,

o similar.

Ejemplo visual:

The image shows a visual example of a product management interface. It is divided into two main sections. The left section contains a list item for a product named 'Mouse Gamer' with a price of '\$20.000'. To the right of the product name and price is a close button represented by an 'X' icon. The right section contains a form for creating a new product. The form has two input fields: the first is labeled 'Nombre' (Name) and the second is labeled 'Precio' (Price). Below these fields is a button labeled 'Crear' (Create).

Sección derecha — Formulario

El formulario estará visible permanentemente.

Debe contener:

Campo nombre.

Campo precio.

Botón crear.

Validación del formulario

El botón deberá permanecer deshabilitado mientras:

- El nombre esté vacío.

- El precio esté vacío.

- El precio sea menor o igual a 0.

Cuando los datos sean válidos:

- El botón se habilitará.

Al hacer clic:

- El producto se agregará a la lista.

- El formulario se limpiará.

Actualización de productos (Opcional)

Quienes deseen realizar actualización deberán implementar el siguiente comportamiento:

Al hacer clic sobre una tarjeta:

- El nombre y precio del producto deben cargarse en el formulario.

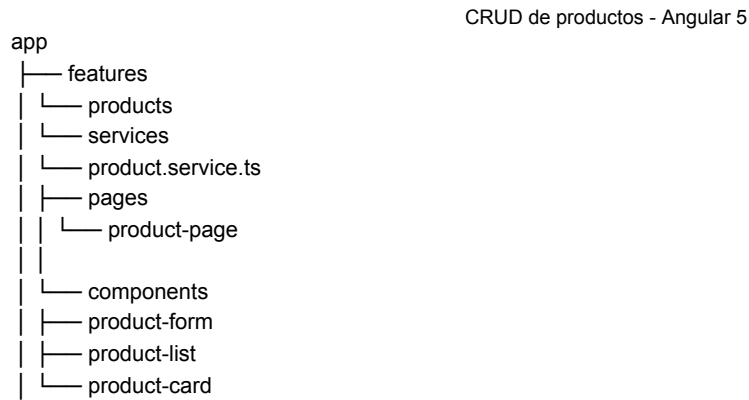
- El botón debe cambiar de Crear a Actualizar

Al actualizar:

- Los cambios deben reflejarse inmediatamente en la lista.

Arquitectura esperada

Se espera una organización similar a:



Requisitos técnicos

Debe existir un servicio

El servicio será responsable de:

Guardar los productos.

Crear productos.

Obtener productos.

Eliminar productos.

(Opcional) Actualizar productos.

⚠ Toda la lógica debe centralizarse en el servicio.

Modelo de producto

Cada producto debe tener:

```
{
  id: number,
  nombre: string,
```

```
precio: number  
}
```

Componentes mínimos

ProductPageComponent

Componente principal que organiza la vista en dos columnas.

ProductFormComponent

Responsable del formulario.

ProductListComponent

Responsable de mostrar:

mensaje vacío,

o lista de productos.

ProductCardComponent

Responsable de representar un producto individual.

Criterios de evaluación

Criterio	Valor
Uso correcto de servicios	25%
Organización modular	20%
Separación por componentes	20%

Funcionamiento del CRUD	20%
Diseño visual y experiencia	10%

CRUD de productos - Angular 7

Criterio	Valor
Buenas prácticas y orden	5%

Penalizaciones

Se penalizará:

- Uso de standalone.
- Uso de tecnologías no vistas en clase.
- Toda la lógica en un solo componente.
- Código desordenado.
- Variables globales.
- Manipulación directa del DOM.
- Uso incorrecto de Angular.

Recomendaciones

Antes de programar:

1. Diseñar la estructura de carpetas.
2. Identificar responsabilidades de cada componente.
3. Crear primero el servicio.
4. Luego construir la interfaz.

Entrega

La entrega debe incluir:

Proyecto completo.

Código funcional.

Interfaz organizada.

CRUD de productos - Angular 8

Buen uso de componentes y servicios.

El proyecto debe ejecutar correctamente con:

`ng serve`

`# o`

`npm start`

